

HW5 — Exercise 23.11 Bouncing Balls

A JavaFX program that bounces balls inside a **Pane**, using the **Timeline** threading technique from Chapter 23.

Behavior (符合評分項目)

1. The **first ball is blue**, sits at the **center** of the pane, and is **stationary**.
2. 「開始」 (**Start**) button: clicking it sends the center blue ball moving in a **random direction**. After the first click the button **greys out (disabled)** and further clicks do nothing — it can only be re-enabled by the 「重來」 button.
3. 「重來」 (**Restart**) button: clicking it resets the pane back to a single stationary blue ball at the center, and re-enables 「開始」.
4. **Clicking the mouse anywhere in the pane** spawns a new ball **from the center** of the pane that immediately moves in a random direction — but only **after** 「開始」 has been pressed (before that, pane clicks do nothing).
5. The pane holds **at most 20 balls**, and the current ball count is shown on screen (目前球數: **N / 20**).
6. Newly spawned balls are **randomly colored** (the blue ball stays blue).

How to compile and run

JavaFX is no longer bundled with the JDK (JDK 11+), so the **lib** folder of a JavaFX SDK must be supplied via the **PATH_TO_FX** variable.

```
make          # compile all .java with the JavaFX module path
make run      # compile (if needed) then launch the window
make clean    # remove all compiled .class files
```

The **Makefile** sets a default path with **?=**, so the grader's environment variable automatically overrides it:

```
PATH_TO_FX ?= C:/javafx-sdk-26.0.1/lib
MODULES     = javafx.controls,javafx.graphics
MAIN        = BouncingBalls
```

javac is invoked with **-encoding UTF-8** so the Chinese button labels compile correctly regardless of the machine's default charset. If your JavaFX SDK is elsewhere, edit that default line or run, e.g.:

```
make run PATH_TO_FX="C:/Users/你的帳號/javafx-sdk-24.0.1/lib"
```

Code overview — **BouncingBalls.java**

A single JavaFX **Application**. A **BorderPane** holds the play area (**pane**) in the center and a control bar (開始, 重來, count label) at the bottom. A list of balls is advanced every animation tick; a ball bounces by negating its velocity when it reaches an edge.

Constants

Field	Value	Description
PANE_WIDTH / PANE_HEIGHT	600 / 400	Size of the play Pane
RADIUS	12	Radius of every ball
SPEED	3.0	Distance (pixels) a moving ball travels per frame
MAX_BALLS	20	Maximum number of balls allowed in the pane
CENTER_X / CENTER_Y	300 / 200	Center of the pane; spawn point of every ball

Instance fields

Field	Type	Description
random	Random	Source of random colors and directions
balls	List<Ball>	All balls currently in the pane
pane	Pane	The play area; receives mouse clicks
startButton	Button	「開始」 — starts the blue ball, then disables itself
restartButton	Button	「重來」 — resets to the initial single blue ball
countLabel	Label	Shows 目前球數: N / 20
started	boolean	Whether 「開始」 has been pressed

Nested class **Ball**

Pairs the visual **Circle** with its velocity:

Member	Type	Description
circle	Circle	The on-screen shape (its center is the ball position)
dx / dy	double	Horizontal / vertical velocity (pixels/frame); 0 means stationary

Methods

Method	Responsibility
start(Stage)	Builds the fixed-size pane , the two buttons and the count label, lays them out in a BorderPane , registers event handlers, calls resetToInitial() , then starts an indefinite Timeline whose KeyFrame fires every 16 ms (~60 fps) to call updateBalls() .

Method	Responsibility
<code>resetToInitial()</code>	Clears all balls and pane children, sets <code>started = false</code> , re-enables 「開始」, and adds the single stationary blue ball at the center (<code>dx = dy = 0</code>).
<code>handleStart()</code>	Ignores repeat clicks; sets <code>started = true</code> , disables 「開始」, and gives the blue ball (<code>balls.get(0)</code>) a random direction via <code>dx = SPEED*cos θ</code> , <code>dy = SPEED*sin θ</code> .
<code>handleRestart()</code>	Calls <code>resetToInitial()</code> .
<code>handlePaneClick()</code>	Does nothing unless <code>started</code> is true and there is room (<code>< MAX_BALLS</code>); otherwise spawns a randomly colored ball at the center moving in a random direction.
<code>addBallAtCenter(color, dx, dy)</code>	Creates a <code>Circle</code> at the center with the given color/velocity, adds it to <code>balls</code> and the pane, and refreshes the count label.
<code>updateCount()</code>	Updates <code>countLabel</code> to 目前球數: <code><size> / 20</code> .
<code>updateBalls()</code>	Called every tick: moves each ball by <code>dx/dy</code> , and on contact with any edge clamps the position back inside and negates the matching velocity component to bounce.
<code>main(String[])</code>	Calls <code>launch(args)</code> to start the JavaFX runtime.

Execution flow (`main` → window)

1. `main` calls `launch`, which starts the JavaFX application thread and invokes `start`.
2. `start` builds the UI, wires the buttons and the pane's mouse handler, calls `resetToInitial` (one blue stationary ball appears at the center), and plays the `Timeline`.
3. Every 16 ms the `Timeline` calls `updateBalls` on the JavaFX Application Thread, so the animation runs while the GUI stays responsive — the Chapter 23 threading technique.
4. Pressing 「開始」 launches the blue ball and disables the button. While started, each pane click spawns a new randomly colored ball from the center (up to 20). Pressing 「重來」 returns everything to the initial single blue ball.

Notes

- The 20-ball cap is enforced in `handlePaneClick`; the count label always reflects the number currently in the pane.
- Random colors use `Color.hsb(hue, 0.8, 0.9)` with a random hue so every spawned ball is bright and distinct.
- Tested with JDK 24.0.1 and JavaFX 26.0.1 (`make`, `make run`, `make clean` all work).